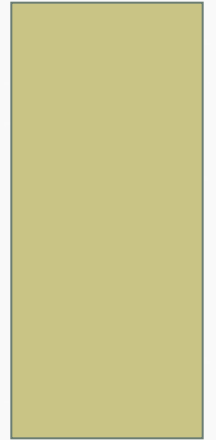


EC-211 DATA STRUCTURES

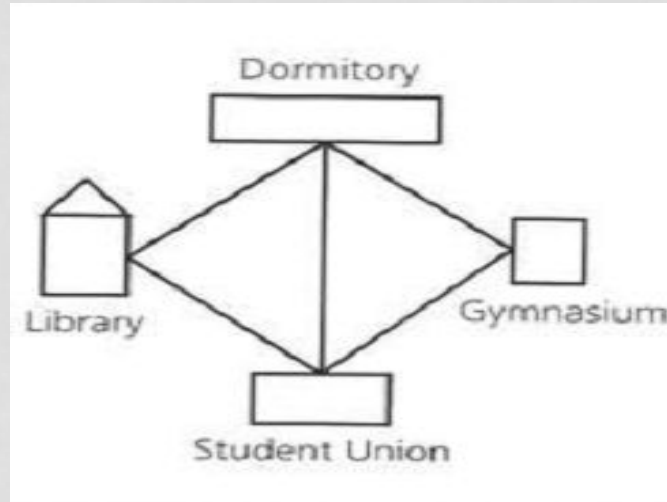
LECTURE 15



GRAPH

- A **graph** is a collection of **nodes** (or **vertices**, singular is **vertex**) and **edges** (or **arcs**)
 - Each node contains an **element**
 - Each edge connects two nodes together (or possibly the same node to itself) and may contain an **edge attribute**
- A **directed graph** is one in which the edges have a direction
- An **undirected graph** is one in which the edges do not have a direction

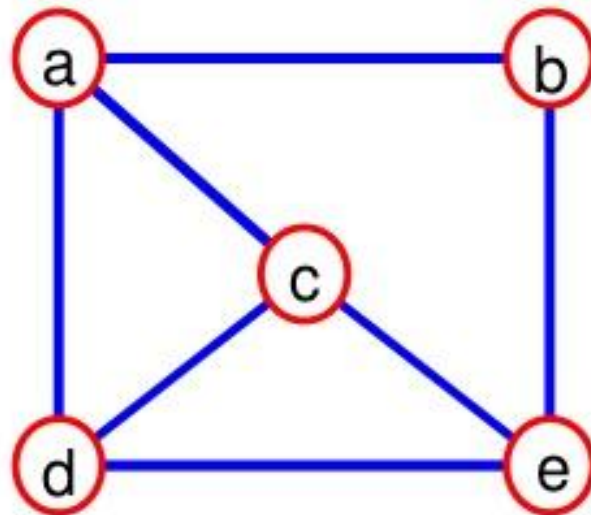
GRAPH



- Graph is a non-linear data structure comprised of two sets:
 - A set of **vertices** V
 - A set of **edges** E
- $G = \{V, E\}$

GRAPH

- A graph $G = (V, E)$ is composed of:
 - V : set of **vertices**
 - E : set of **edges** connecting the **vertices** in V
- An **edge** $e = (u, v)$ is a pair of **vertices**
- Example:

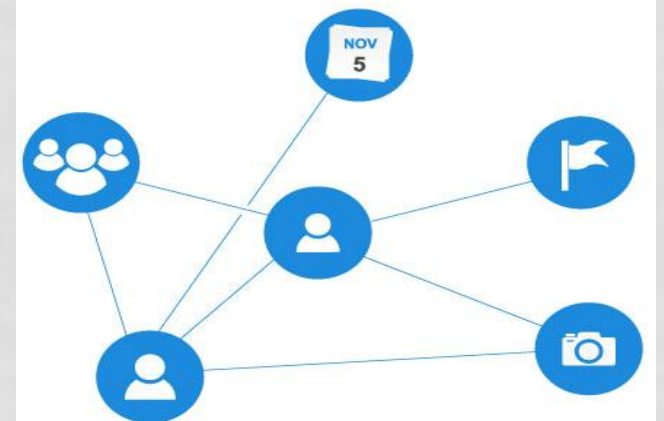
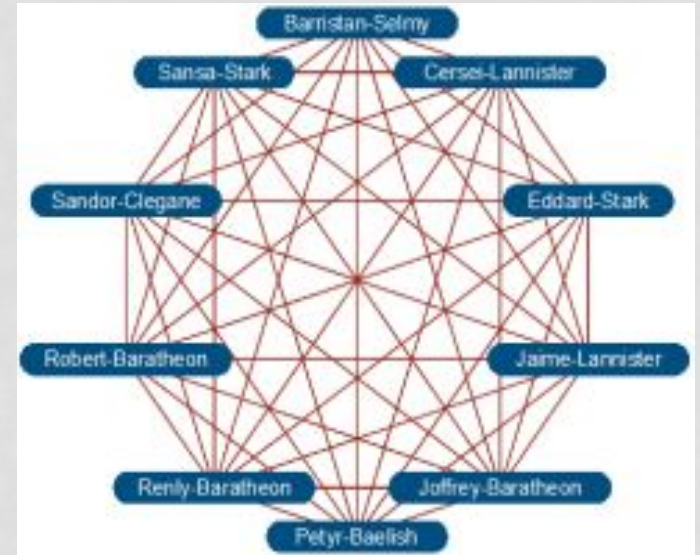


$V = \{a, b, c, d, e\}$

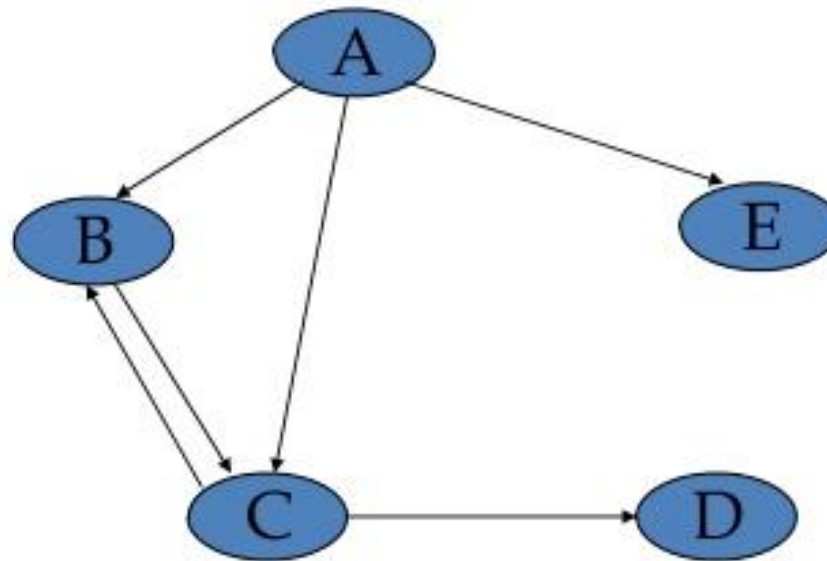
$E = \{(a, b), (a, c), (a, d), (b, e), (c, d), (c, e), (d, e)\}$

APPLICATIONS

- computer networks
- airline flights
- road map
- course prerequisite structure
- tasks for completing a job
- flow of control through a program



DIRECTED GRAPH



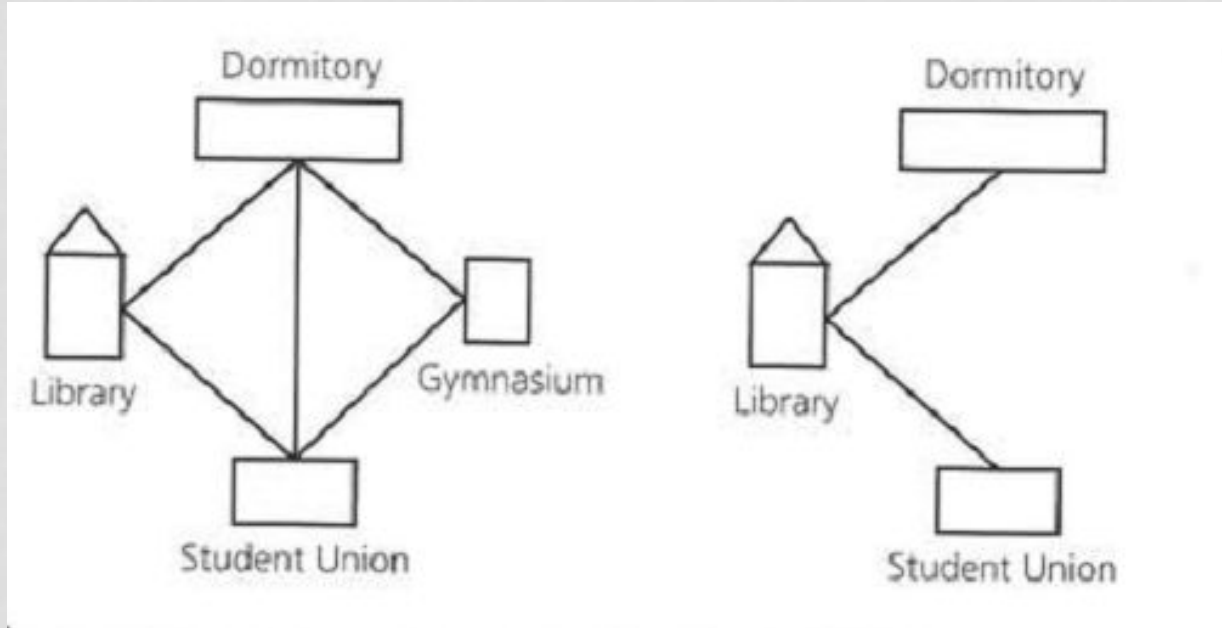
$V = [A, B, C, D, E]$

$E = [\langle A, B \rangle, \langle B, C \rangle, \langle C, B \rangle, \langle A, C \rangle, \langle A, E \rangle, \langle C, D \rangle]$

TERMINOLOGY

- The **size** of a graph is the number of *nodes* in it
- The **empty graph** has size zero (no nodes)
- If two nodes are connected by an edge, they are **neighbors** (and the nodes are **adjacent** to each other)
- The **degree of a node** is the number of edges it has
- For directed graphs,
 - If a directed edge goes from node *S* to node *D*, we call *S* the **source** and *D* the **destination** of the edge
 - The edge is an **out-edge** of *S* and an **in-edge** of *D*
 - *S* is a **predecessor** of *D*, and *D* is a **successor** of *S*
 - The **in-degree** of a node is the number of in-edges it has
 - The **out-degree** of a node is the number of out-edges it has

TERMINOLOGY



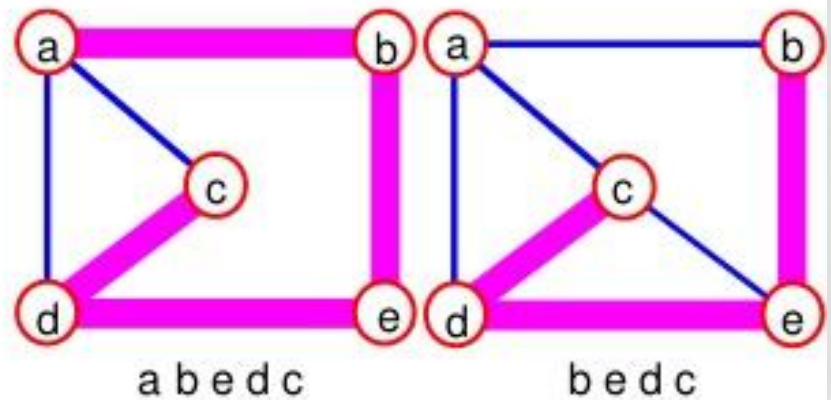
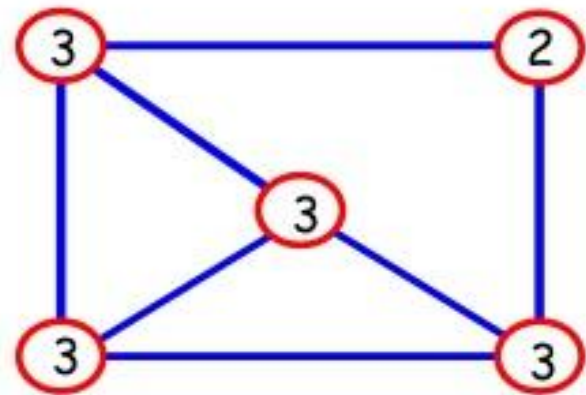
- **Subgraph:** A subset of vertices and edges of the graph
- **Adjacent Vertices:** Two vertices that are joined by an edge

TERMINOLOGY

- **Path:** A path between two vertices is a sequence of edges
- **Simple Path:** A path in which each vertex appears only once
- **Simple Cycle:** A path that begins and ends at the same vertex. Rest of the vertices appear only once
- **Cycle:** A path that begins and ends at the same vertex.

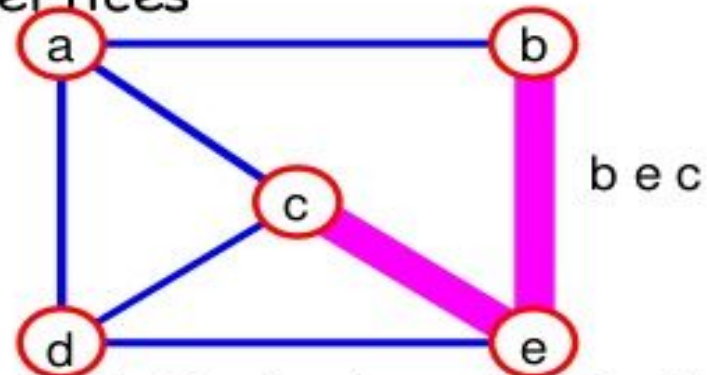
TERMINOLOGY

- **path**: sequence of vertices $v_1, v_2, \dots, v_k$ such that consecutive vertices v_i and v_{i+1} are adjacent.

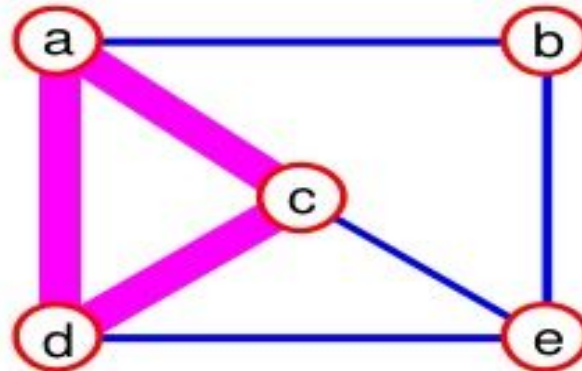


TERMINOLOGY

- **simple path:** no repeated vertices



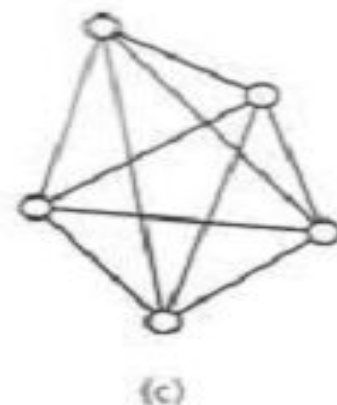
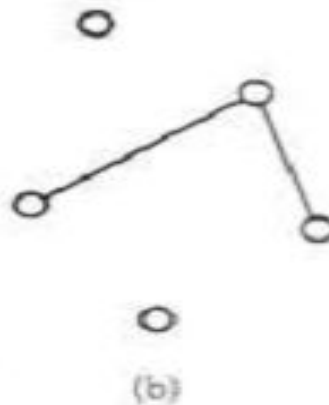
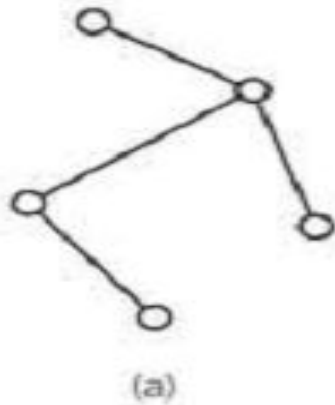
- **cycle:** simple path, except that the last vertex is the same as the first vertex



a c d a



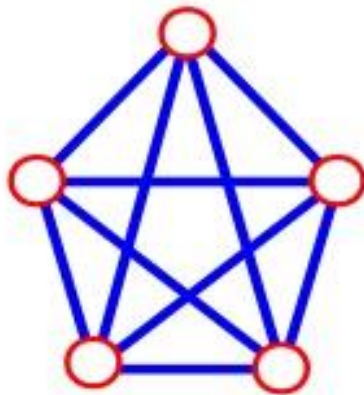
CONNECTED, DISCONNECTED, AND COMPLETE GRAPHS



- A connected graph has a **path** between every pair of distinct vertices
- A complete graph has an **edge** between every pair of distinct vertices

COMPLETE GRAPH

- Let $n = \text{\#vertices}$, and $m = \text{\#edges}$
- **A complete graph**: one in which all pairs of vertices are adjacent
- *How many total edges in a complete graph?*
 - Each of the n vertices is incident to $n-1$ edges, however, we would have counted each edge twice! Therefore, intuitively, $m = n(n-1)/2$.
- Therefore, if a graph is not complete, $m < n(n-1)/2$



$$\begin{aligned} n &= 5 \\ m &= (5 * 4)/2 = 10 \end{aligned}$$

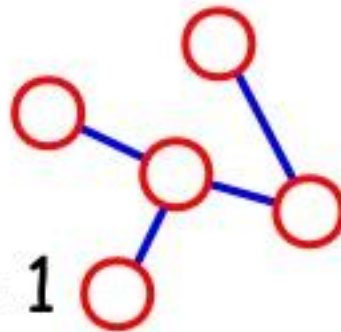
CONNECTED GRAPH

n = #vertices

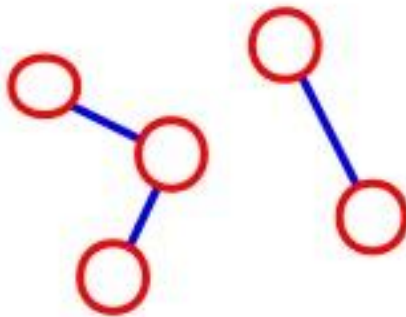
m = #edges

- For a tree $m = n - 1$

If $m < n - 1$, G is not connected



$n = 5$
 $m = 4$



$n = 5$
 $m = 3$

DIRECTED VS UNDIRECTED GRAPH

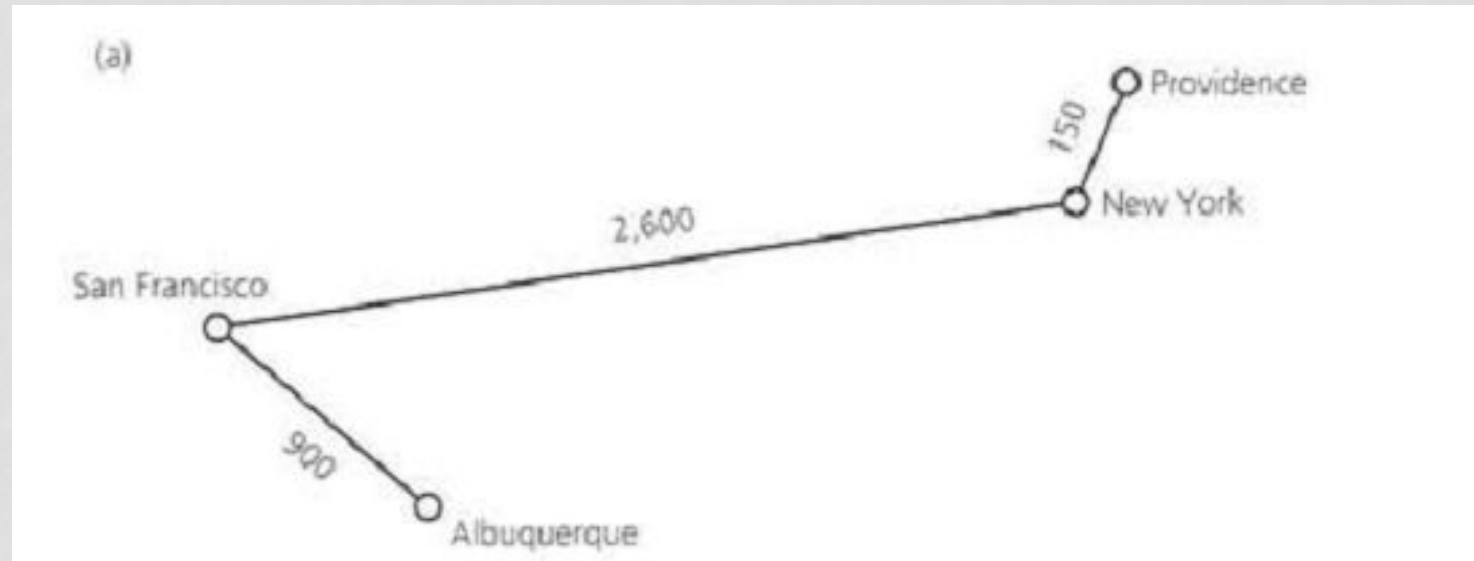
- An **undirected graph** is one in which the pair of vertices in a edge is unordered, $(v_0, v_1) = (v_1, v_0)$
- A **directed graph** is one in which each edge is a directed pair of vertices, $\langle v_0, v_1 \rangle \neq \langle v_1, v_0 \rangle$



TERMINOLOGY

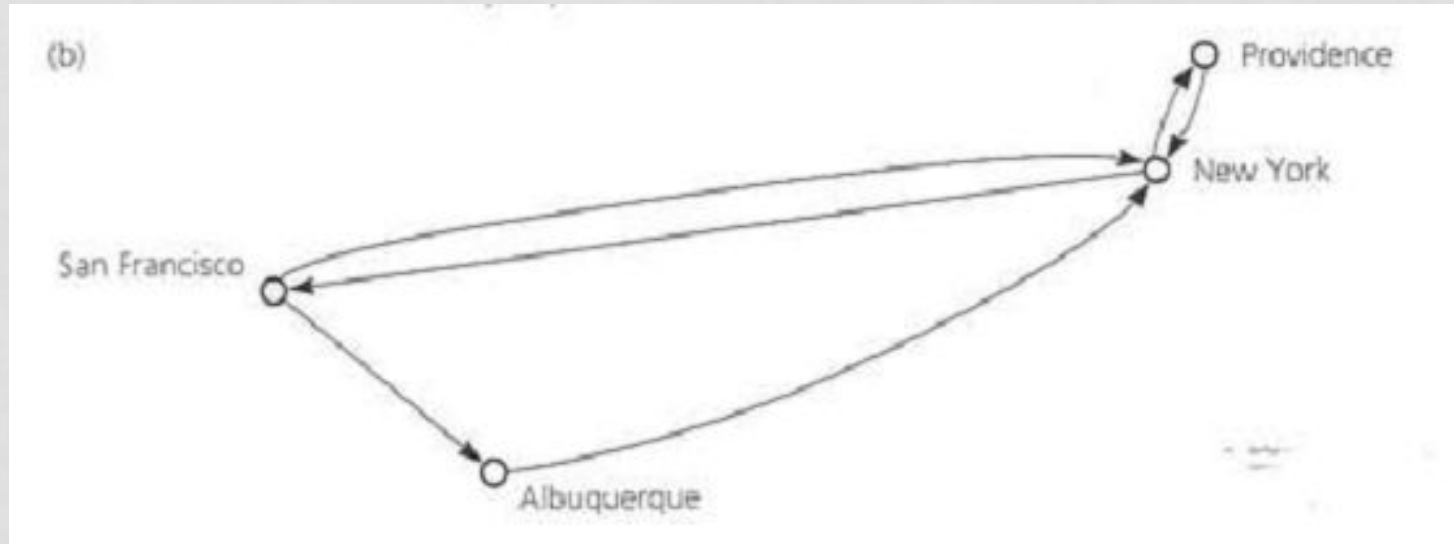
- An undirected graph is **connected** if there is a path from every node to every other node
- A *directed graph* is **strongly connected** if there is a path from every node to every other node
- A directed graph is **weakly connected** if the underlying undirected graph is connected
- Node **X** is **reachable** from node **Y** if there is a path from **Y** to **X**
- A subset of the nodes of the graph is a **connected component** (or just a **component**) if there is a path from every node in the subset to every other node in the subset

WEIGHTED GRAPH



- The edges of a weighted graph have numeric labels

DIRECTED GRAPH (DIGRAPH)



- Each edge in a directed graph has a direction

IMPLEMENTING GRAPHS

- **Adjacency Matrix**
 - Adjacency matrix of a graph with n vertices $0, 1, \dots, n-1$ is an $n \times n$ array matrix such that $\text{matrix}[i][j]$ is 1 (true) if there is an edge from i to j , and is 0 (false) otherwise.
- **Adjacency List**
 - Adjacency list of a graph with n vertices $0, 1, \dots, n-1$ consists of n linked lists. The i th linked list contains a node for vertex j if and only if the graph contains an edge from i to j

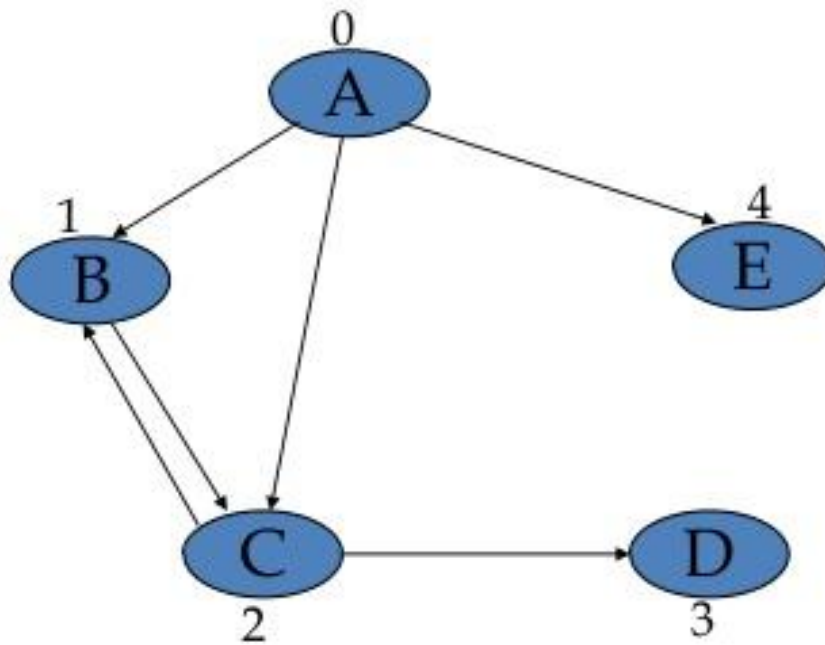
IMPLEMENTATION

- storing the vertices
 - each vertex has a unique identifier and, maybe, other information
 - for efficiency, associate each vertex with a number that can be used as an index
- storing the edges
 - adjacency matrix - represent all possible edges
 - adjacency lists - represent only the existing edges

REPRESENTING VERTEX

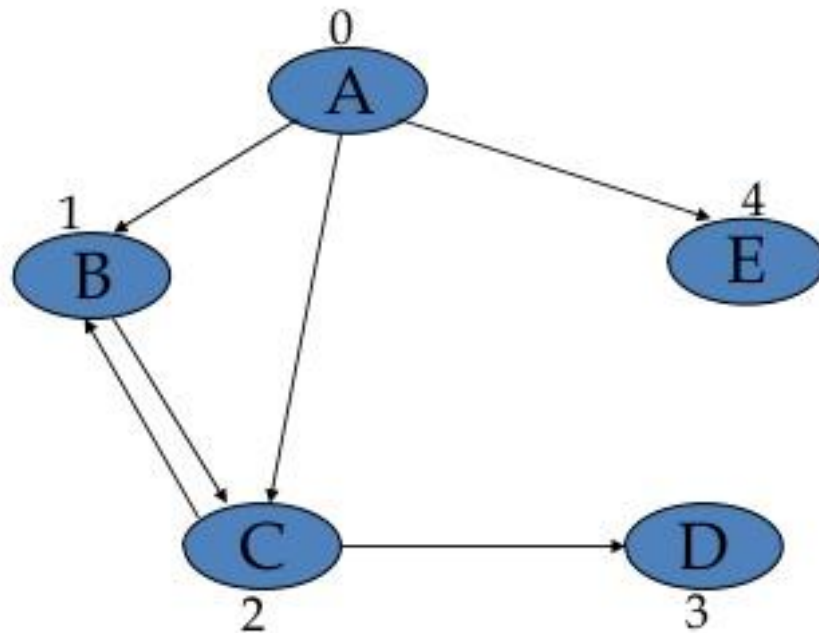
- when a vertex is added to the graph, assign it a number
 - vertices are numbered between 0 and $n-1$
- graph operations start by looking up the number associated with a vertex
- many data structures to use
 - any of the associative data structures
 - for small graphs a vector can be used
 - search will be $O(n)$

VERTEX VECTOR



0	A
1	B
2	C
3	D
4	E

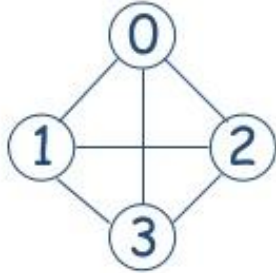
ADJACENCY MATRIX



0	A
1	B
2	C
3	D
4	E

	0	1	2	3	4
0	0	1	1	0	1
1	0	0	1	0	0
2	0	1	0	1	0
3	0	0	0	0	0
4	0	0	0	0	0

ADJACENCY MATRIX



G_1

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

symmetric

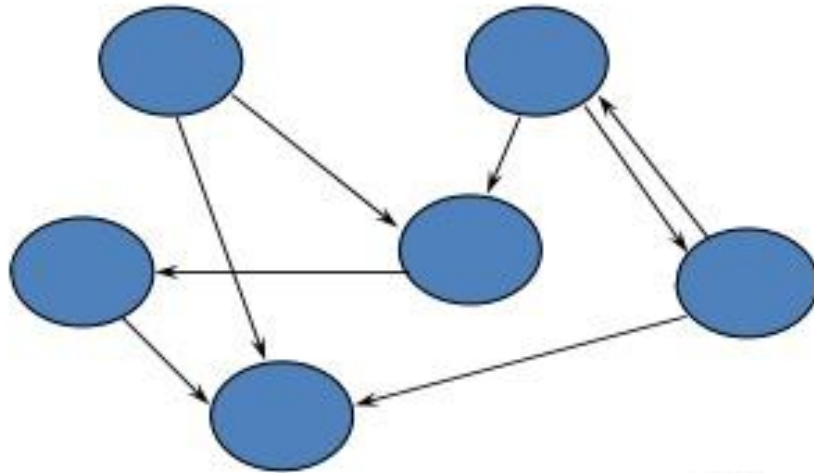


G_2

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

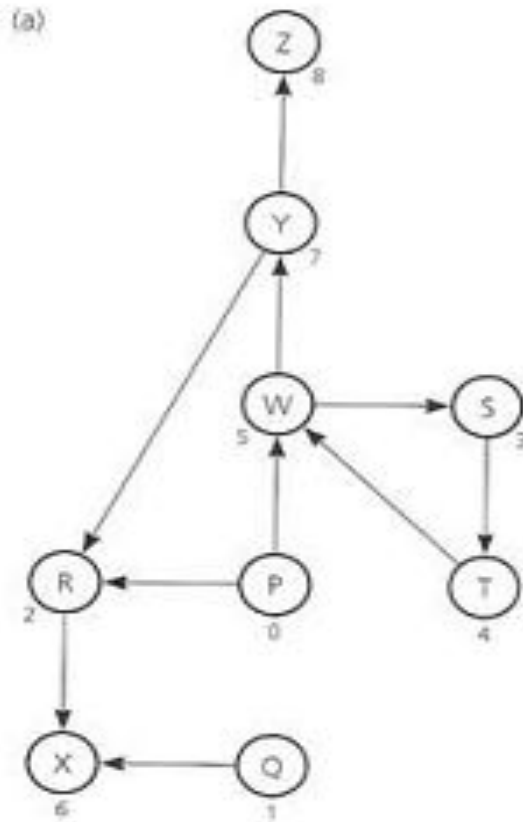
Asymmetric

SPACE COMPLEXITY?



a V^2 matrix is needed for
a graph with V vertices

ADJACENCY MATRIX OF A DIRECTED (UNWEIGHTED) GRAPH



(b)

		0	1	2	3	4	5	6	7	8
		P	Q	R	S	T	W	X	Y	Z
0	P	0	0	1	0	0	1	0	0	0
1	Q	0	0	0	0	0	0	1	0	0
2	R	0	0	0	0	0	0	1	0	0
3	S	0	0	0	0	1	0	0	0	0
4	T	0	0	0	0	0	1	0	0	0
5	W	0	0	0	1	0	0	0	1	0
6	X	0	0	0	0	0	0	0	0	0
7	Y	0	0	1	0	0	0	0	0	1
8	Z	0	0	0	0	0	0	0	0	0

FIGURE 13-6

(a) A directed graph and (b) its adjacency matrix

ADJACENCY MATRIX OF A DIRECTED (WEIGHTED)GRAPH

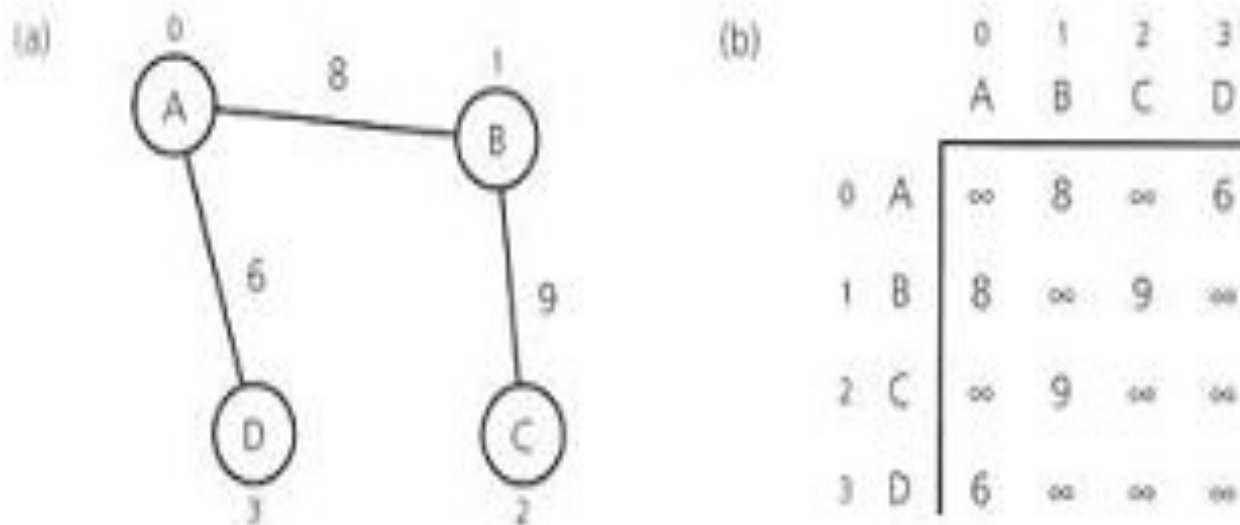
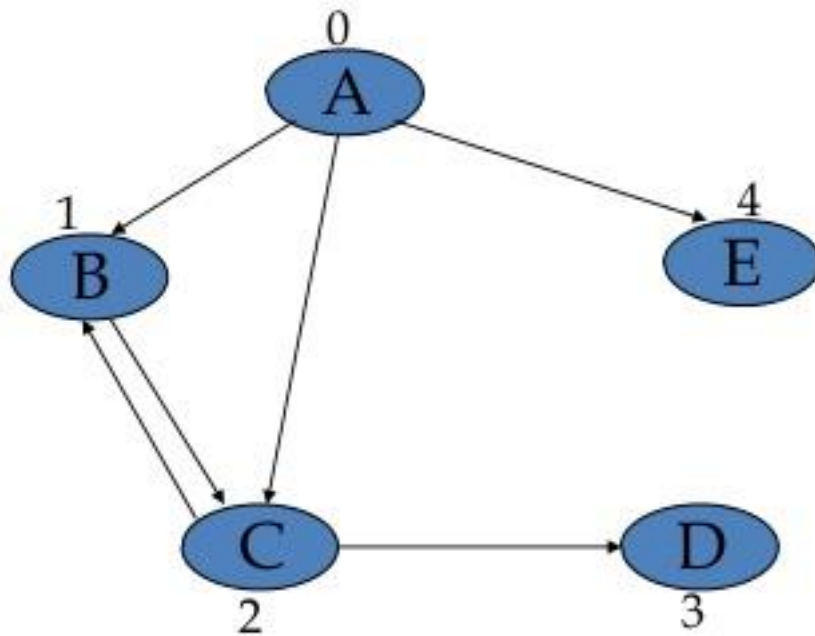


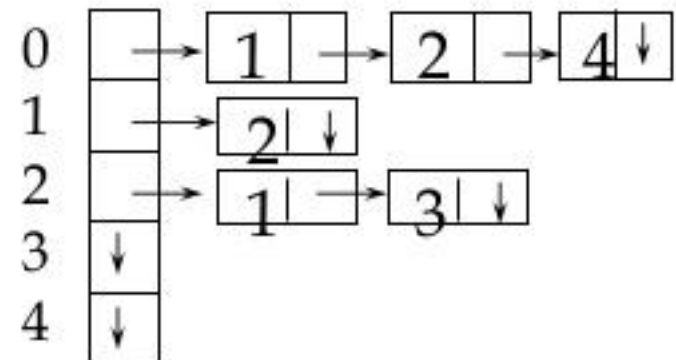
FIGURE 13-7

(a) A weighted undirected graph and (b) its adjacency matrix

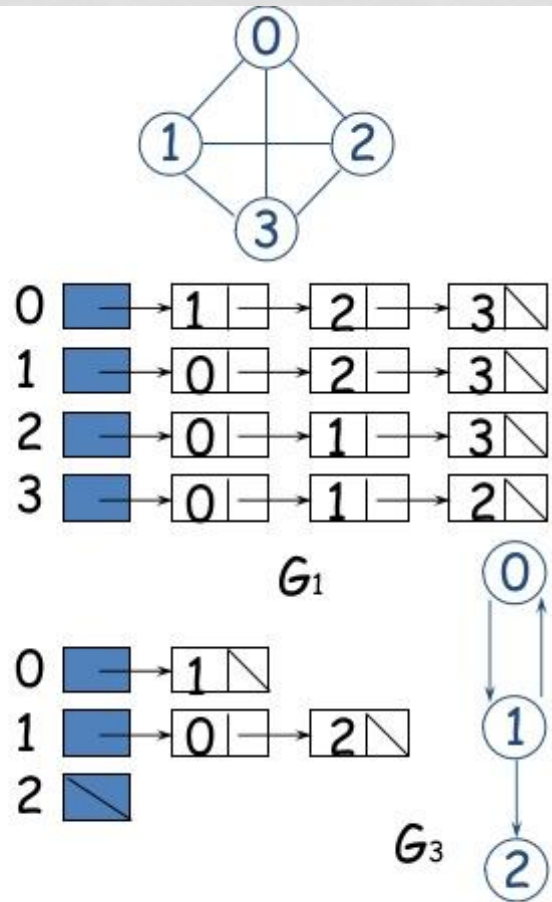
ADJACENCY LIST



0	A
1	B
2	C
3	D
4	E



ADJACENCY LIST



ADJACENCY LIST OF A DIRECTED (UNWEIGHTED)GRAPH

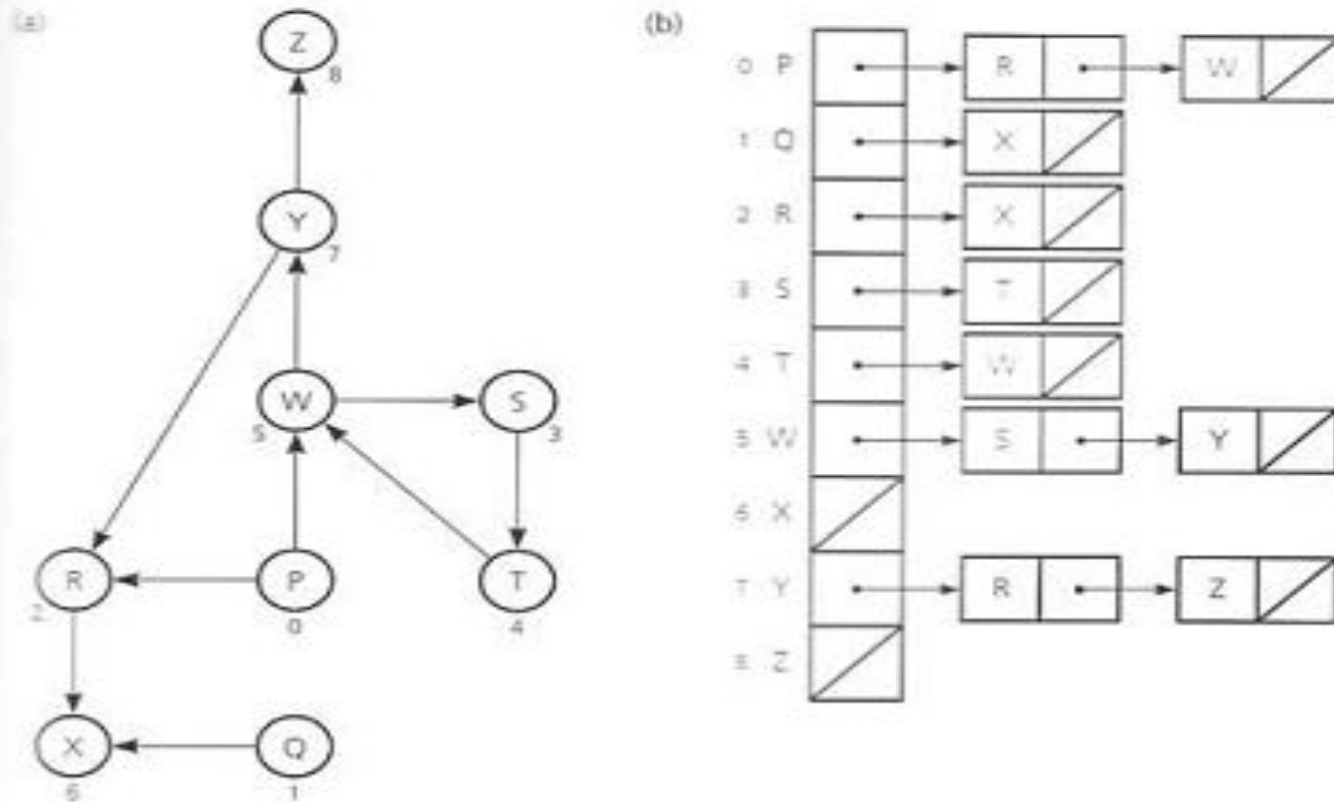
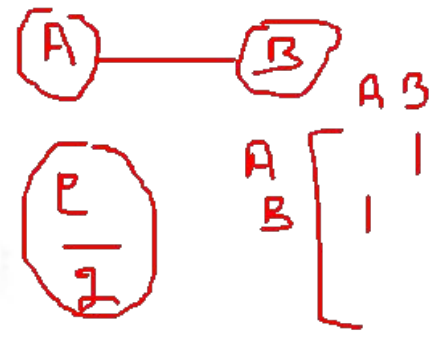
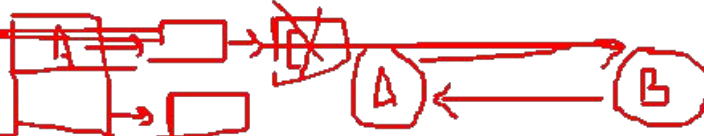


FIGURE 13-8

(a) A directed graph and (b) its adjacency list

ADT Graph Operations

1. Create an empty graph.
2. Destroy a graph.
3. Determine whether a graph is empty.
4. Determine the number of vertices in a graph.
5. Determine the number of edges in a graph.
6. Determine whether an edge exists between two given vertices.
7. Insert a vertex in a graph whose vertices have distinct search keys that differ from the new vertex's search key.
8. Insert an edge between two given vertices in a graph.
9. Delete a particular vertex from a graph and any edges between the vertex and other vertices.
10. Delete the edge between two given vertices in a graph.
11. Retrieve from a graph the vertex that contains a given search key.



COMPARISON

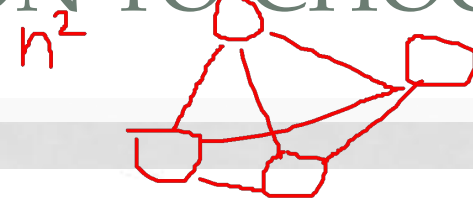
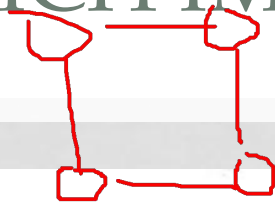
- Adjacency matrix

- Answer “does edge $[i][j]$ exist?” in $O(1)$.
- Space used: $O(N^2)$ where N = number of vertices.
- Finding all neighbors of a vertex can be slow for large, sparsely connected graphs.

- Adjacency list

- Answer “does edge $[i][j]$ exist?” in $O(N)$.
- Much better space usage for large, sparse graphs.
- Finding all neighbors of a vertex is fast.

WHICH IMPLEMENTATION TO CHOOSE?



- degree of "sparseness" key factor in choosing a data structure for edges
 - adjacency matrix requires space for **all possible** edges
 - adjacency list requires space for existing edges only
- affects amount of memory space needed
- affects efficiency of graph operations

GRAPH TRAVERSALS — INTRODUCTION

- **Depth-first search (DFS).**
 - Like preorder tree traversal.
 - Goes as far as possible from a vertex before backing up
 - When we visit a vertex, give priority to visiting its unvisited neighbors (and their unvisited neighbors, etc.).
- **Breadth-first search (BFS).**
 - Visit all of a vertex's unvisited neighbors before visiting their neighbors.

GRAPH TRAVERSALS

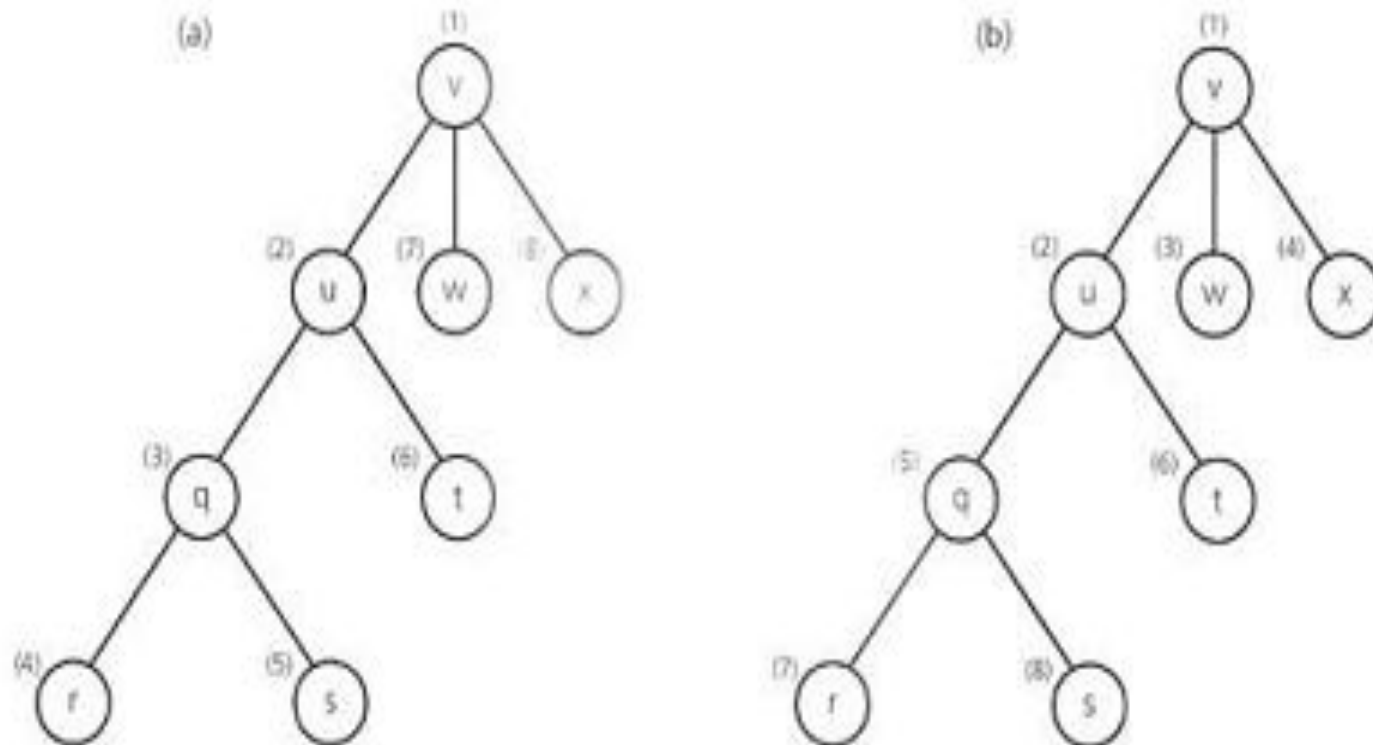


FIGURE 13-10

Visitation order for (a) a depth-first search; (b) a breadth-first search

DFS ALGORITHM

```
dfs(in v:Vertex)
// Traverses a graph beginning at vertex v by using a
// depth-first search: Iterative version.

    s.createStack()

    // push v onto the stack and mark it
    s.push(v)
    Mark v as visited

    // Loop invariant: there is a path from vertex v at the
    // bottom of the stack s to the vertex at the top of s
    while (!s.isEmpty())
    {
```

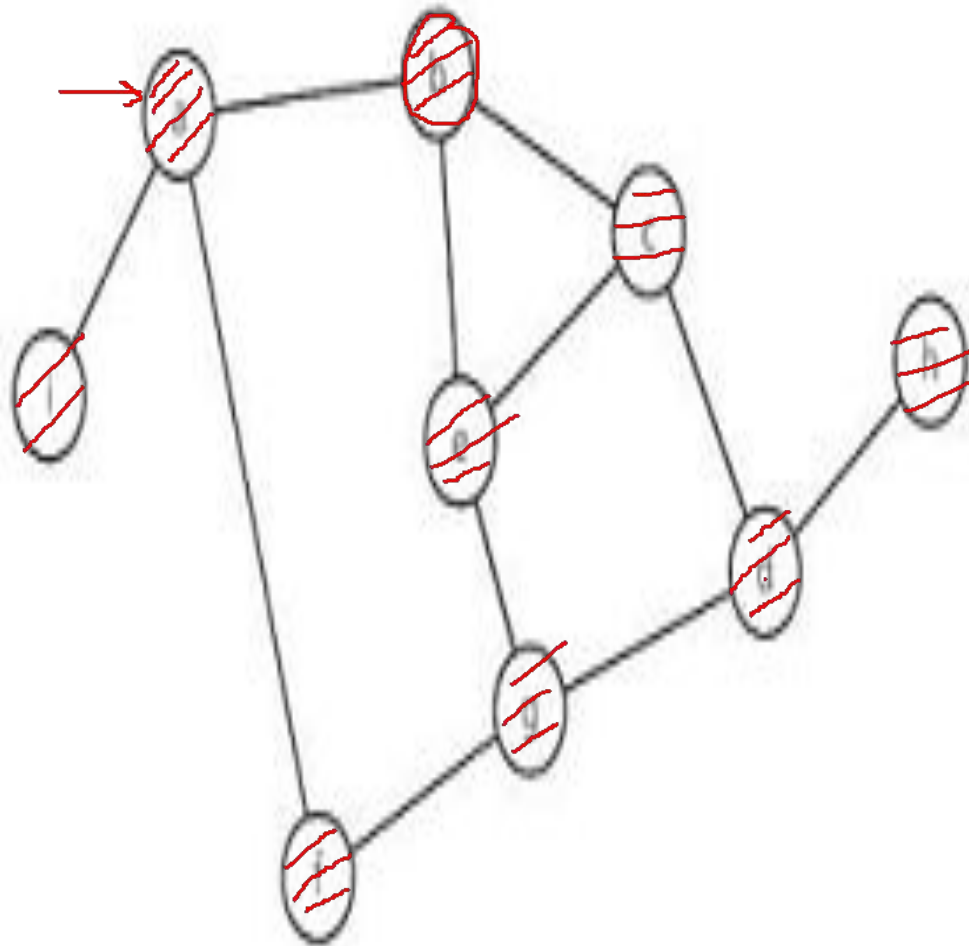
DFS ALGORITHM CONTD.

```
if (no unvisited vertices are adjacent to
    the vertex on the top of the stack)
    s.pop()  // backtrack

else
{   Select an unvisited vertex u adjacent to
    the vertex on the top of the stack
    s.push(u)
    Mark u as visited
}   // end if
}   // end while
```

DFS EXAMPLE

efghdcba



Node visited	Stack (bottom to top)
a	<u>a</u>
b	a b
c	a b c
d	a b c d
e	a b c d e
(backtrack)	a b c d <u>g</u>
f	a b c d g f
(backtrack)	a b c d g
(backtrack)	a b c d <u>h</u>
h	a b c d h
(backtrack)	a b c d
(backtrack)	a b c
(backtrack)	a b
(backtrack)	a
i	a <u>i</u>
(backtrack)	a
(backtrack)	(empty)

BFS ALGORITHM

```
bfs(in v:Vertex)
// Traverses a graph beginning at vertex v by using a
// breadth-first search: Iterative version.

    q.createQueue()

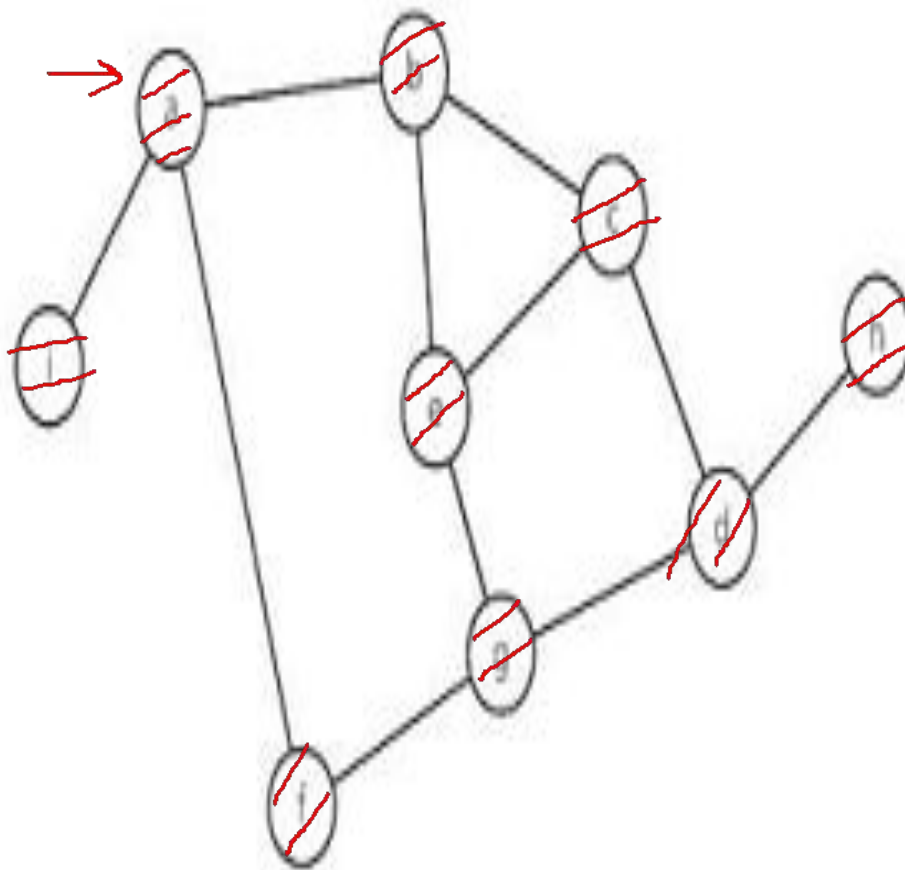
    // add v to queue and mark it
    q.enqueue(v)
    Mark v as visited
```

BFS ALGORITHM CONTD.

```
while (!q.isEmpty())  
{  q.dequeue(w)  
  
    // Loop invariant: there is a path from vertex w to  
    // every vertex in the queue q  
    for (each unvisited vertex u adjacent to w)  
    {  Mark u as visited  
        q.enqueue(u)  
    }  // end for  
}  // end while
```

BFS EXAMPLE

a b f i c e g d h



Node visited

a

b

f

i

c

e

g

d

h

Queue (front to back)

a

(empty)

b

b f

b f i

f i

f i c

f i c e

i c e

i c e g

→ c e g

e g

e g d

g d

d

(empty)

→ h

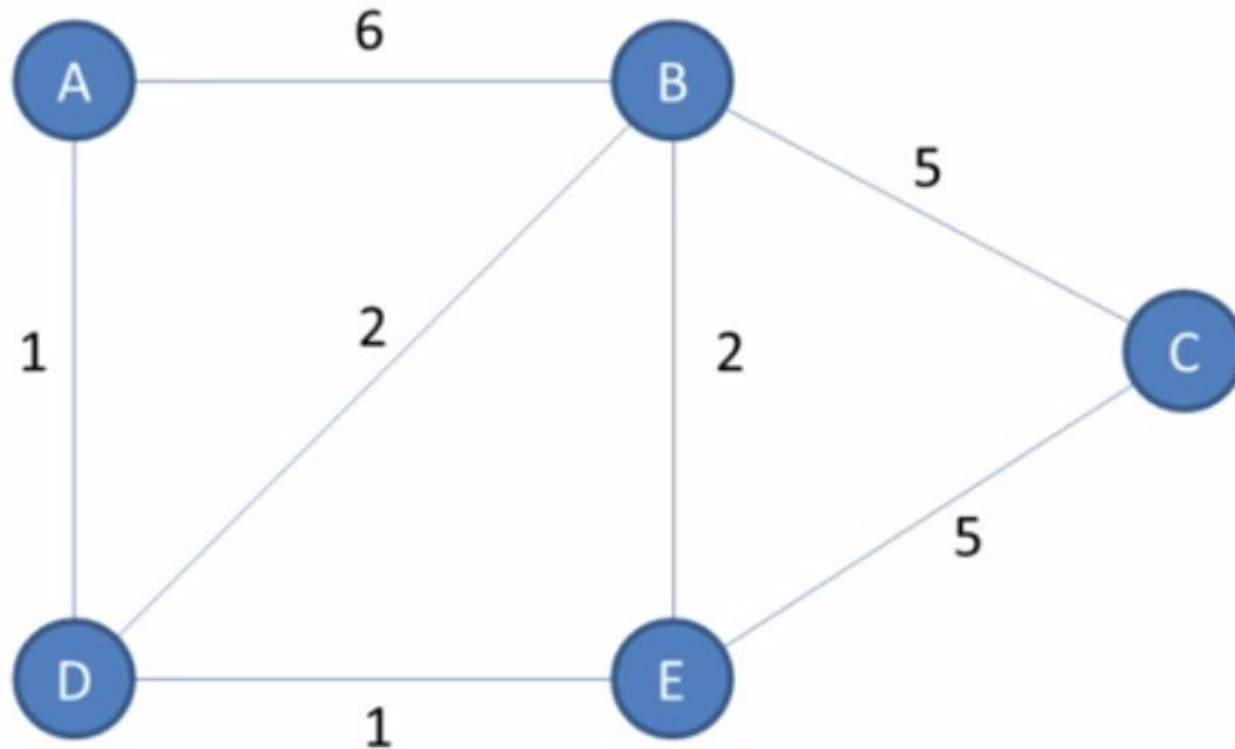
(empty)

SHORTEST PATH IN WEIGHTED GRAPH – DIJKSTRA'S ALGORITHM

```
vertexSet = {}, Parent[ ] = none for all nodes  
C[start] = 0, C[ ] =  $\infty$  for all other nodes  
while ( not all nodes in vertexSet )  
    find node v not in vertexSet with smallest C[v]  
    add v to vertexSet  
    for each node J not in vertexSet adjacent to v  
        if ( C[v] + cost of (v,J) < C[J] )  
            C[J] = C[v] + cost of (v,J)  
            Parent[J] = v
```

***Optimal* solution computed with *greedy* algorithm**

DIJKSTRA'S ALGORITHM: EXAMPLE

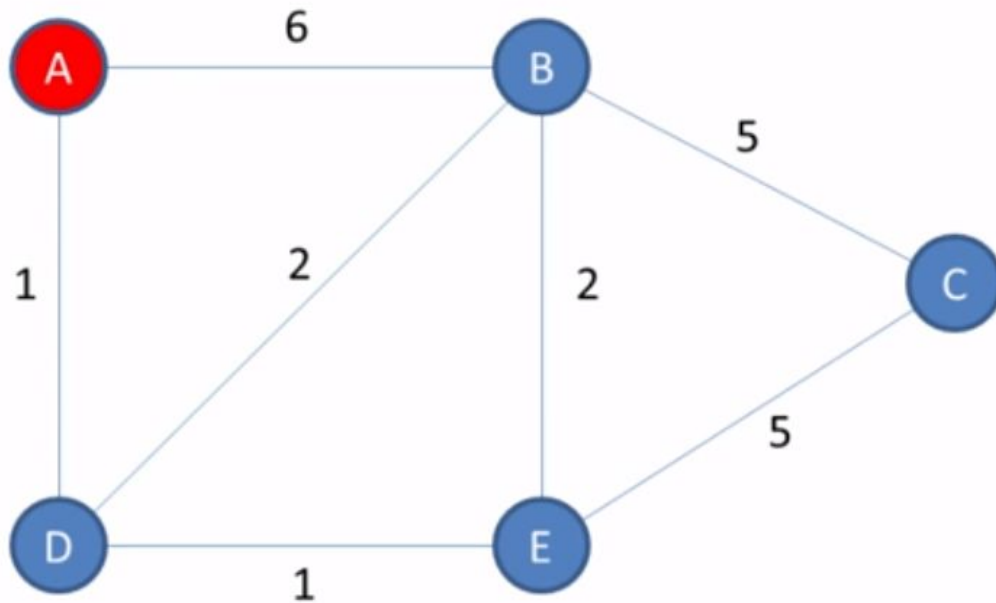


DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Consider the start vertex, A

Distance to A from A = 0

Distances to all other vertices from A are unknown, therefore ∞ (infinity)



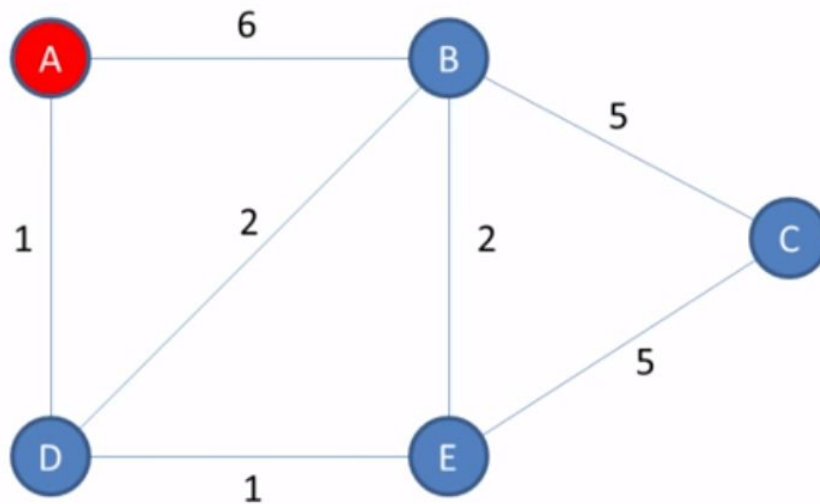
Vertex	Shortest distance from A	Previous vertex
A		
B		
C		
D		
E		

Visited = []

Unvisited = [A, B, C, D, E]

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Visit the unvisited vertex with the smallest known distance from the start vertex
First time around, this is the start vertex itself, A



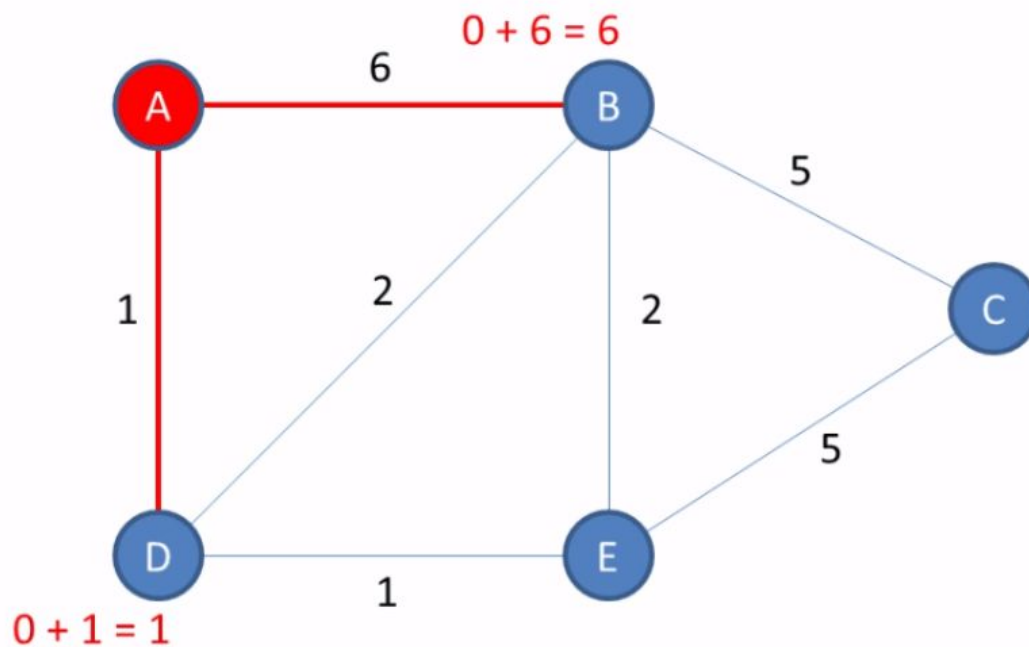
Visited = []

Unvisited = [A, B, C, D, E]

Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Update the previous vertex for each of the updated distances
In this case we visited B and D via A



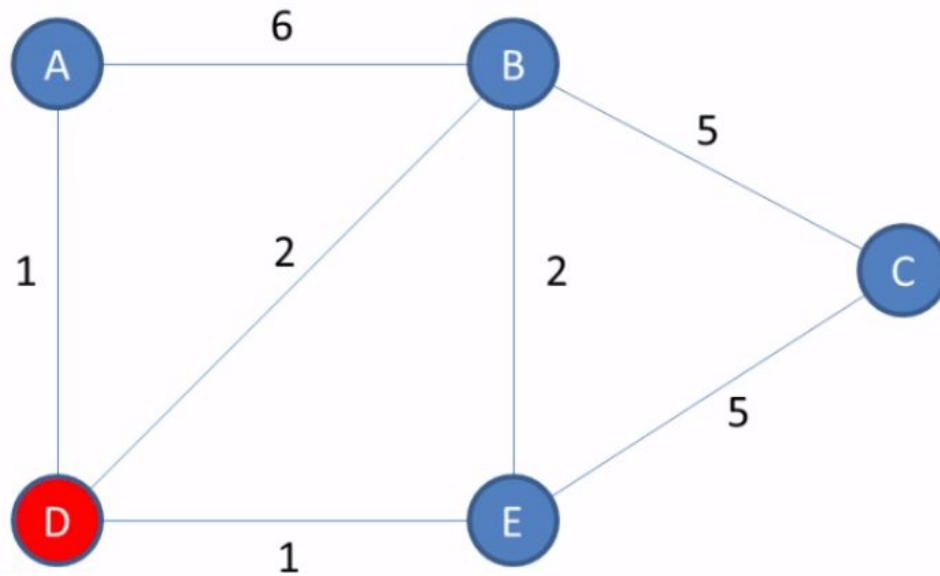
Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	
C	∞	
D	1	
E	∞	

Visited = []

Unvisited = [A, B, C, D, E]

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex D



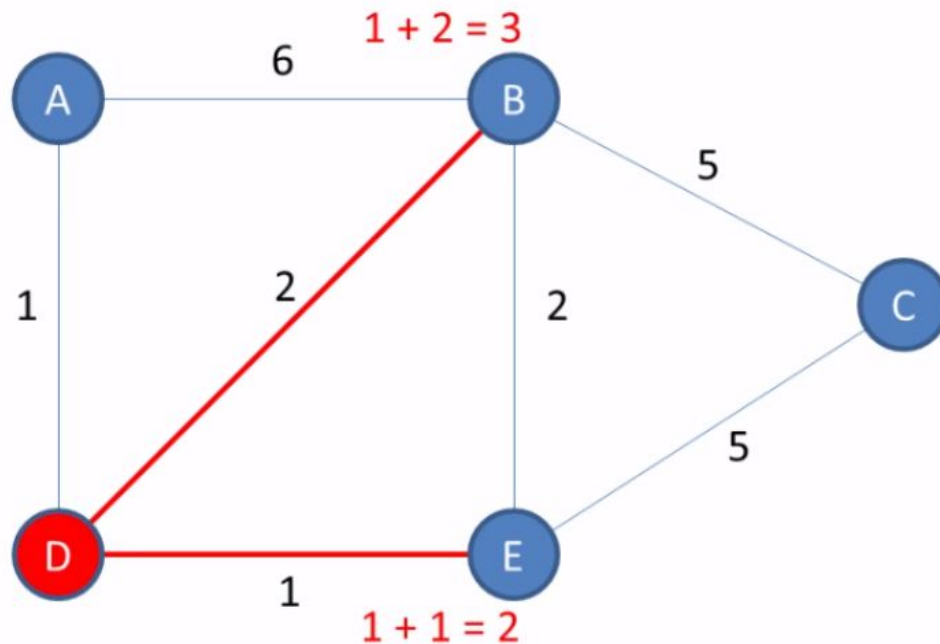
Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	A
C	∞	
D	1	A
E	∞	

Visited = [A]

Unvisited = [B, C, D, E]

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Update the previous vertex for each of the updated distances
In this case we visited B and E via D



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	∞	
D	1	A
E	2	D

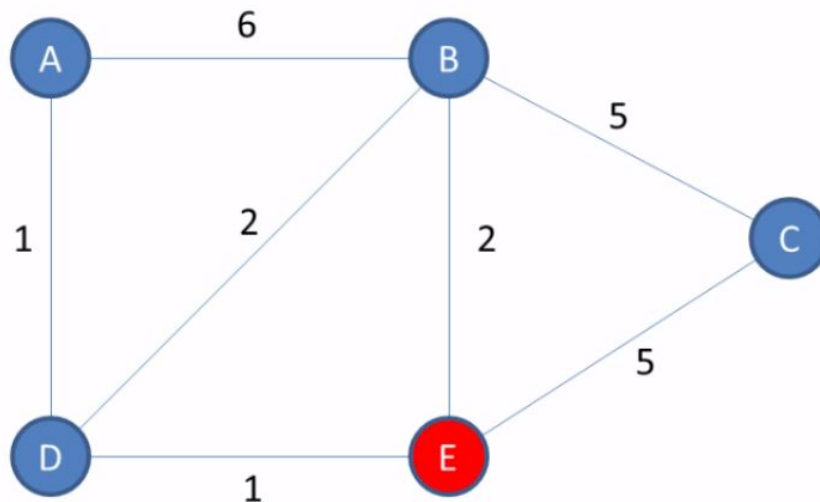
Visited = [A]

Unvisited = [B, C, D, E]

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Visit the unvisited vertex with the smallest known distance from the start vertex

This time around, it is vertex E



Visited = [A, D]

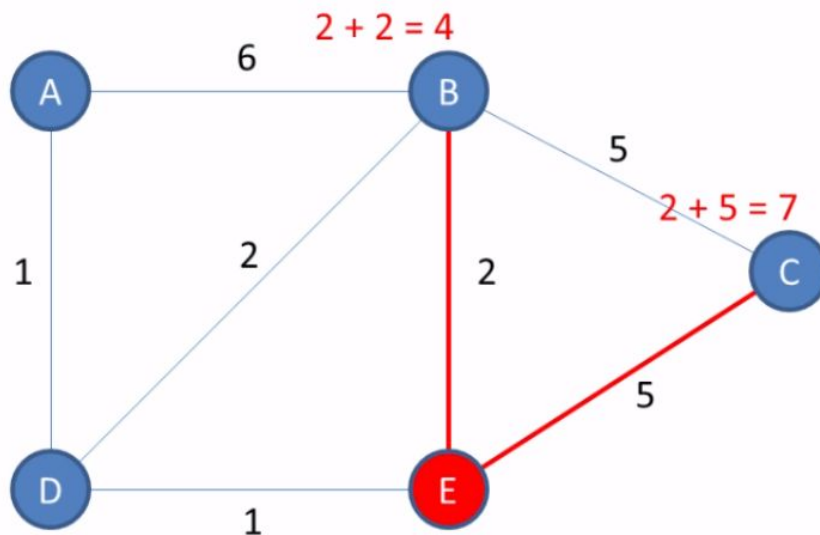
Unvisited = [B, C, E]

Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	∞	
D	1	A
E	2	D

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Update the previous vertex for each of the updated distances

In this case we visited C via E



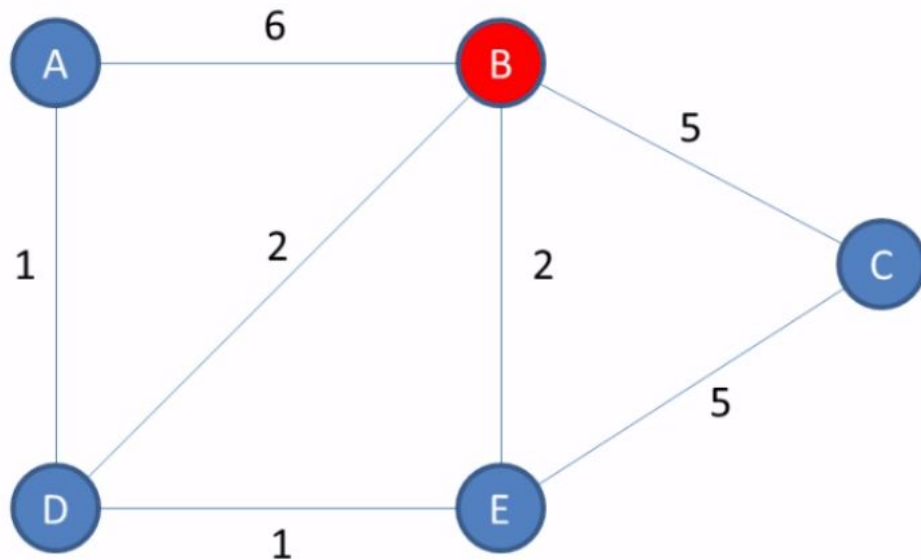
Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D]

Unvisited = [B, C, E]

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex B



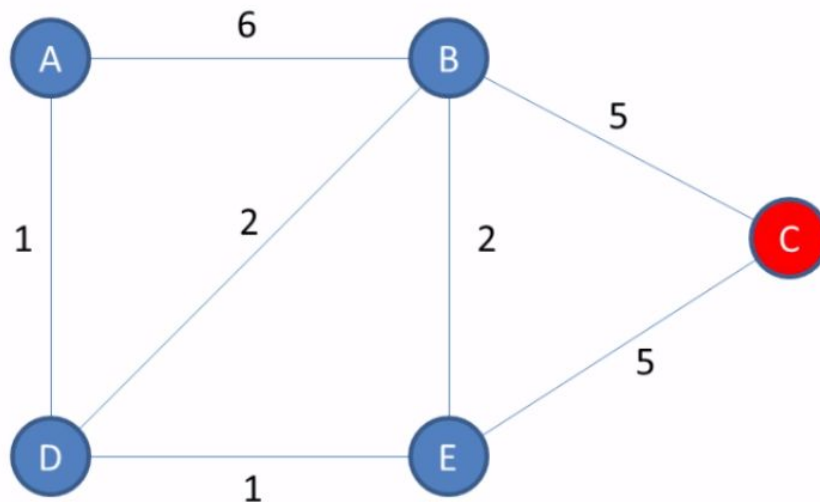
Visited = [A, D, E]

Unvisited = [B, C]

Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

DIJKSTRA'S ALGORITHM: EXAMPLE CONTD.

Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex C



Visited = [A, D, E, B]

Unvisited = [C]

Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

C - E - D - A

READING

- Chapter 20, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- <https://leapgraph.com/graph-data-structures-applications>